Rational Decisions and Classical Machine Learning

Undergraduate lecture • Theory, mathematical intuition, real-world cases and Python

DATA → PREDICTION → UTILITY → ACTION

How intelligent systems learn from evidence and choose what to do

Learning pathway

01

Rational agents

Actions and goals

02

Probability

Represent uncertainty

03

Expected utility

Compare consequences

04

Machine learning

Predict from data

05

Decision policy

Turn predictions into actions

Central question: How can an AI system act well when outcomes are uncertain?

OBJECTIVES

What students will learn

- Define rational action and distinguish it from a guaranteed successful outcome
- Compute expected utility and decision thresholds
- Explain Bayesian updating and decision-making under uncertainty
- Understand regression, classification, trees, forests, and clustering
- Use predictive probabilities to choose cost-effective actions

KEY INSIGHT

A model that predicts accurately is not automatically a system that makes good decisions.

Rationality: choose the best expected action

- An agent perceives the environment and chooses an action
- Available evidence may be incomplete or noisy
- The rational choice maximizes expected performance
- An unfavorable outcome does not prove the choice was irrational

KEY INSIGHT

Rationality is judged using information available BEFORE the action, not hindsight.

Agent–environment interaction

01

Sensors

Observe temperature, images, or other data

02

Belief state

Estimate what may be happening

03

Decision rule

Evaluate available actions

04

Actuators

Brake, move, maintain, or alert

The agent repeats this loop as new information arrives.

Rationality vs omniscience

RATIONAL AGENT

- Uses evidence available now
- Handles uncertainty explicitly
- Can still experience failure
- Evaluated by expected outcomes

OMNISCIENT AGENT

- Knows actual future outcomes
- Has no uncertainty about events
- Always knows what will occur
- Not realistic in most AI tasks

The rational action rule

$$a^* = \arg\max_a \; E[\, U \mid a, e \,]$$

SYMBOLS & MEANING

- a^*: selected action
- a: candidate action
- U: utility of outcome
- e: evidence currently available
- $E[\cdot]$: probability-weighted expectation

WORKED INTERPRETATION

For a robot, candidate actions may be “short route” or “safe route.” Select the one with the higher expected utility after considering travel time and collision risk.

Example: autonomous delivery robot

01

Observe

Obstacle detection is uncertain

02

Predict

Estimate collision probability

03

Value

Assign cost to delay and collision

04

Choose

Select highest expected utility route

Shortest path ≠ best decision when safety and uncertainty matter.

PART 2

Why probabilities alone are insufficient

- A 90% success rate is not enough to judge an action
- Success and failure may have very different consequences
- Utility assigns a preference score to outcomes
- Rational choice combines probability AND utility

KEY INSIGHT

High probability of success can still be unattractive when failure is catastrophic.

Expected utility

$$EU(a) = \Sigma_i P(o_i \mid a) \times U(o_i)$$

SYMBOLS & MEANING

- EU(a): expected utility of action a
- o_i: one possible outcome
- $P(o_i|a)$: probability of that outcome
- $U(o_i)$: value assigned to outcome
- Σ: sum across mutually exclusive outcomes

WORKED INTERPRETATION

Multiply each outcome's utility by its probability, then add the products. This produces the probability-weighted average desirability of the action.

Worked example: Project A

$$EU(A) = 0.9 \times 100 + 0.1 \times (-20) = 88$$

SYMBOLS & MEANING

- Success probability = 0.90
- Success utility = +100
- Failure probability = 0.10
- Failure utility = −20

WORKED INTERPRETATION

The successful outcome contributes +90 utility points. The failure outcome contributes −2 points. The total expected utility is 88.

Worked example: Project B

$$EU(B) = 0.6 \times 200 + 0.4 \times (-50) = 100$$

SYMBOLS & MEANING

- Success probability = 0.60
- Success utility = +200
- Failure probability = 0.40
- Failure utility = −50

WORKED INTERPRETATION

Success contributes +120; failure contributes −20. Project B has a LOWER success probability but HIGHER expected utility.

Which project is rational under MEU?



Project B wins under the stated utility scale, despite being riskier.

Risk preferences

RISK NEUTRAL

- Maximizes expected monetary value
- Utility is linear in money
- Averages dominate decisions

RISK AVERSE

- Dislikes large losses and uncertainty
- Utility is typically concave
- May prefer lower-variance outcomes

Maximum Expected Utility (MEU)

$$a^* = \arg\max_a EU(a)$$

SYMBOLS & MEANING

- arg max: the action, not the maximum value
- a: an available decision
- EU(a): expected utility of action a

WORKED INTERPRETATION

Compute EU for every available action, then choose the largest. If actions are restricted by safety rules, maximize only over feasible actions.

Python: compare expected utilities

```
actions = {
 'A': ([0.9, 0.1], [100, -20]),
 'B': ([0.6, 0.4], [200, -50])
}

def EU(probs, utilities):
    return sum(p*u for p,u in
               zip(probs, utilities))

scores = {name: EU(*values)
          for name, values in actions.items()}
print(scores)
print(max(scores, key=scores.get))
```

WHAT THE CODE DOES

1. Represent outcomes and utilities.

2. Multiply probabilities by utilities.

3. Sum expected utility for each action.

4. Select the maximum: Project B.

Decisions under uncertainty

- The true state may be hidden
- Sensors and measurements may be noisy
- Probability represents uncertainty about events
- New observations should update beliefs

KEY INSIGHT

Belief updating and action selection are separate but connected tasks.

Bayes' theorem

$$P(H \mid D) = P(D \mid H) \times P(H) / P(D)$$

SYMBOLS & MEANING

- H: hypothesis (e.g., failure)
- D: new evidence (e.g., alarm)
- P(H): prior belief
- P(D|H): likelihood
- P(D): evidence probability
- P(H|D): updated posterior

WORKED INTERPRETATION

The numerator combines prior belief with compatibility of evidence. The denominator P(D) normalizes the result so probabilities remain valid.

Evidence probability P(D)

$$P(D) = P(D|H)P(H) + P(D|\neg H)P(\neg H)$$

SYMBOLS & MEANING

- H and ¬H: mutually exclusive possibilities
- P(D|H): true-positive likelihood
- P(D|¬H): false-positive likelihood
- P(H)+P(¬H)=1

WORKED INTERPRETATION

Evidence can occur whether H is true or false. P(D) includes BOTH pathways; omitting false positives can greatly overestimate posterior risk.

Bayesian belief to rational action

01

Prior

Initial probability of failure

02

Observation

A vibration alarm arrives

03

Posterior

Update failure probability

04

Utility

Compare maintenance costs

05

Action

Maintain or continue

Bayesian inference tells us what to believe; decision theory tells us what to do.

Expected cost of continuing

$$C_{continue} = p_{failure} \times C_{failure}$$

SYMBOLS & MEANING

- p_failure: predicted failure probability
- C_failure: loss if failure occurs
- Assume no other cost if no failure
- C_continue: expected monetary loss

WORKED INTERPRETATION

If p = 0.30 and failure costs $10,000, expected cost is $3,000. Compare it against the cost of maintenance.

Optimal maintenance threshold

Maintain if p_failure > C_maint / C_failure

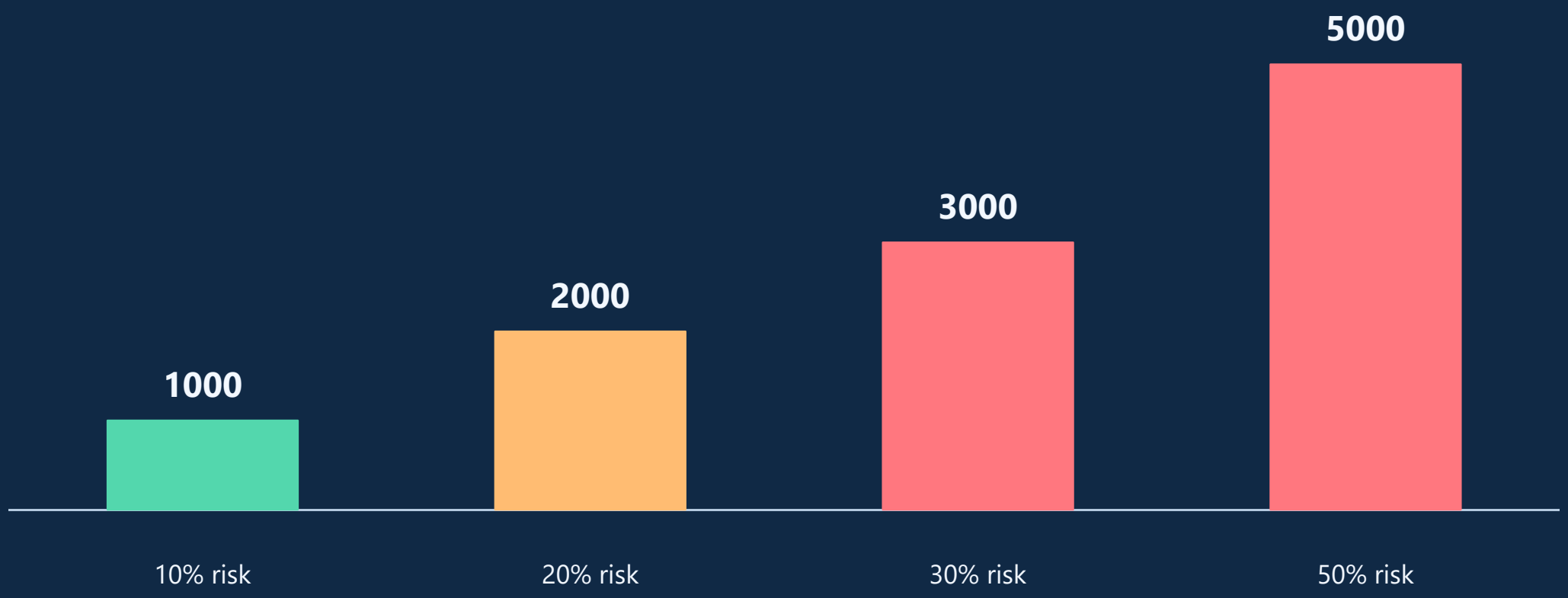
SYMBOLS & MEANING

- C_maint: preventive maintenance cost
- C_failure: cost of an unprevented failure
- Assume maintenance prevents failure fully
- Threshold = 2,000 / 10,000 = 0.20

WORKED INTERPRETATION

A 20% threshold follows from COSTS, not from model accuracy. If failure is more expensive, the optimal threshold becomes lower.

Decision threshold example



Expected failure costs ($); maintenance costs $2,000. Above 20%, maintenance is preferable.

Why move beyond classical programming?

A smooth transition: explicit rules → learning patterns from examples

1 Classical programming

Humans write the rules

```
IF temperature > 80:
    trigger alarm
```

2 The limitation

Real data are noisy
Thresholds may overlap
Patterns can be nonlinear

3 Machine learning

Examples + outcomes
Algorithm learns a model
Model predicts new cases

Key shift: instead of programming every decision rule, we learn useful rules from data.

The same problem, two approaches

Machine condition monitoring: rules written by a person vs a model learned from data

RULE-BASED

Temperature + vibration

Human-defined threshold

Alarm / no alarm

DATA-DRIVEN ML

Temperature + vibration

Learned classifier

Failure probability

Both approaches can be useful; ML is especially valuable when patterns are difficult to specify manually.

Traditional programming vs ML

TRADITIONAL PROGRAMMING

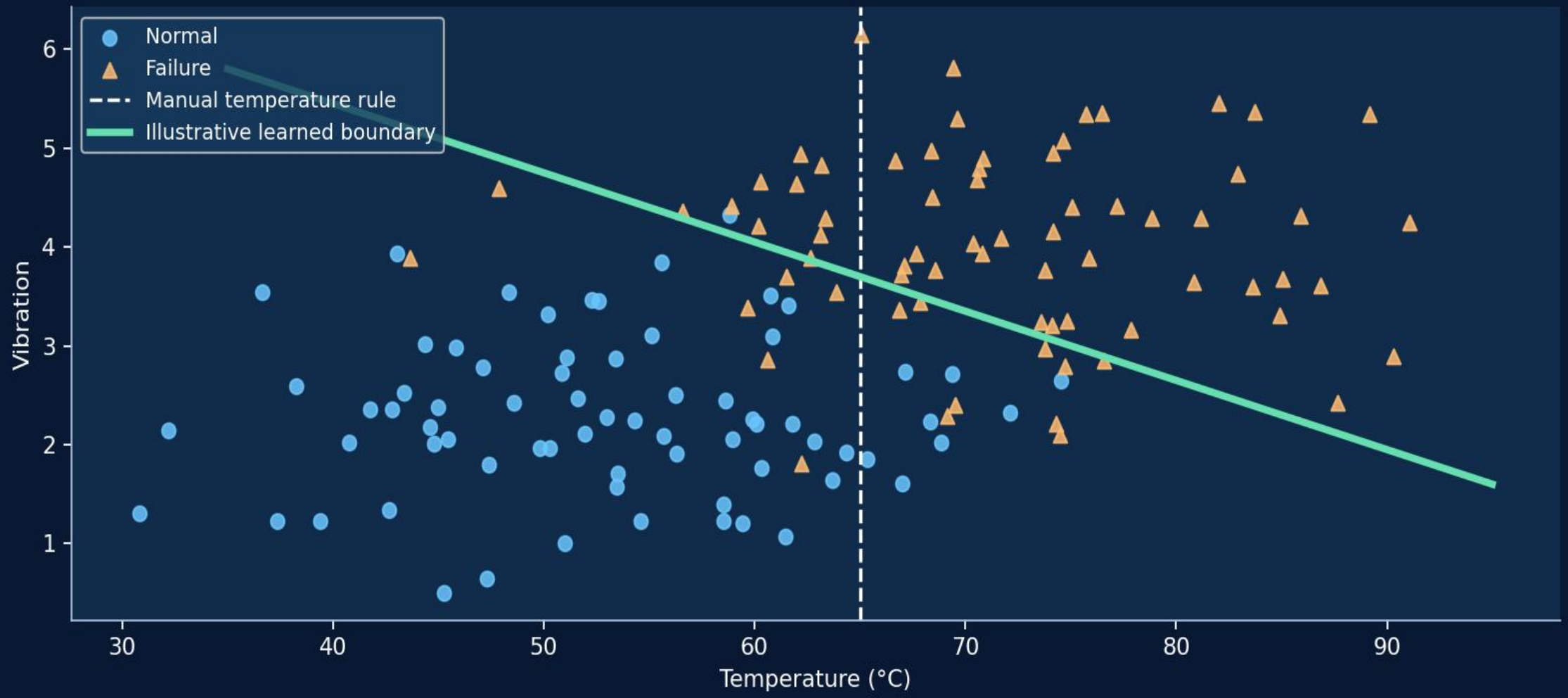
- Human writes explicit rules
- Data + rules → output
- Easy to audit for simple cases

MACHINE LEARNING

- Algorithm learns from examples
- Data + outcomes → fitted model
- Useful for complex patterns

Visual transition: rules vs learned boundaries

A hand-written threshold cannot always separate overlapping machine conditions



A predictive model

$$\hat{y} = f\theta(x)$$

SYMBOLS & MEANING

- x: input features
- θ: learned model parameters
- fθ: mapping from inputs to predictions
- ŷ: model output (number or class)

WORKED INTERPRETATION

Example: x = [temperature, vibration]. The learned model estimates a future failure probability using these features.

Machine learning workflow

01

Collect

Gather labeled examples

02

Split

Separate training and test data

03

Fit

Learn model parameters

04

Evaluate

Measure unseen-data performance

05

Deploy

Predict on new observations

Prevent leakage: test information must not influence model fitting.

Three major machine learning paradigms

The key difference is the feedback available during learning

SUPERVISED

Learn from labeled examples
Input → known answer



UNSUPERVISED

Find patterns without labels
Input → hidden structure



REINFORCEMENT

Learn by trial and feedback
Action → reward



Examples: failure classification • customer segmentation • robot navigation

How do the three types differ?

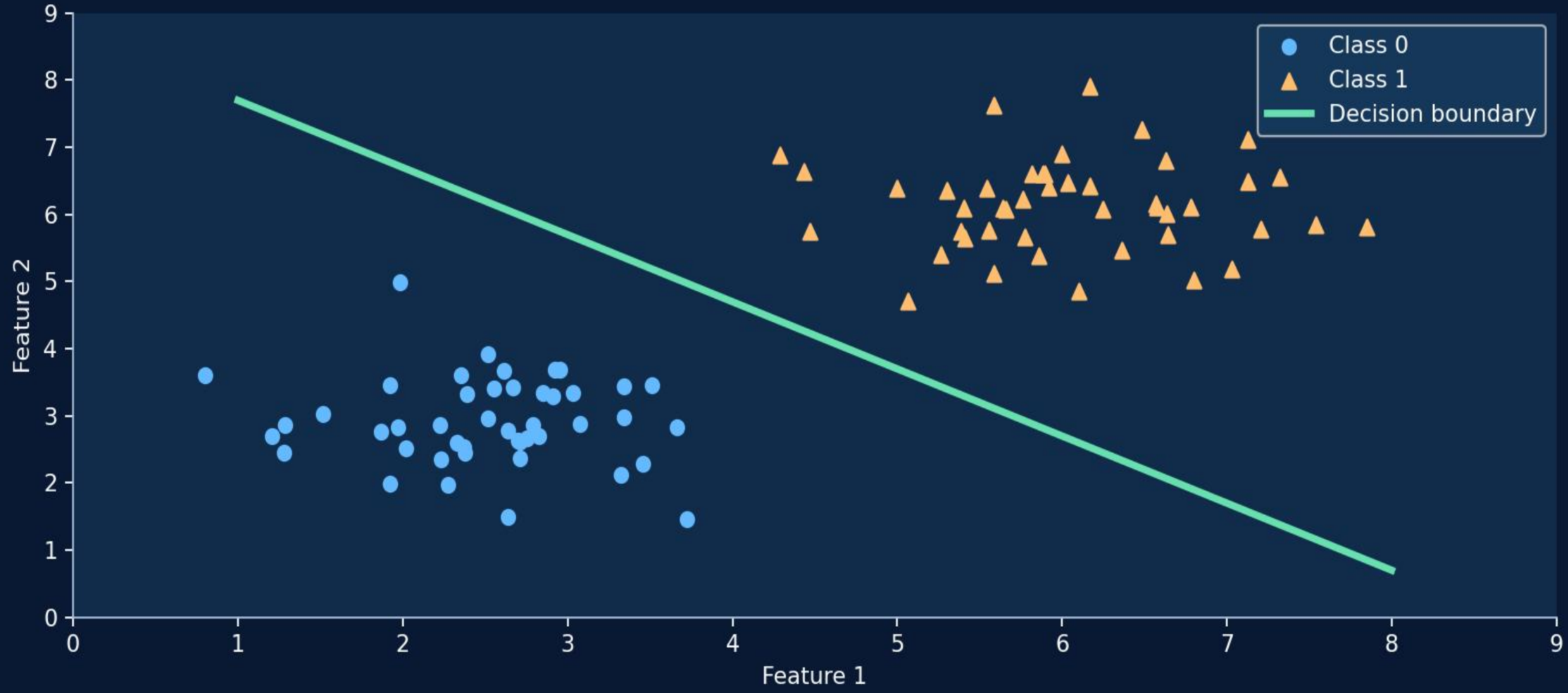
Compare what the learner receives and what it tries to discover

Dimension	Supervised	Unsupervised	Reinforcement
Training signal	Correct labels	No labels	Rewards / penalties
Main objective	Predict target	Discover structure	Maximize total reward
Typical output	Class or number	Clusters / components	Action policy
Examples	Failure detection	Customer groups	Robot control
Algorithms	Regression, trees	K-Means, PCA	Q-learning, DQN

Important: reinforcement learning is not the same as unsupervised learning — rewards guide behavior.

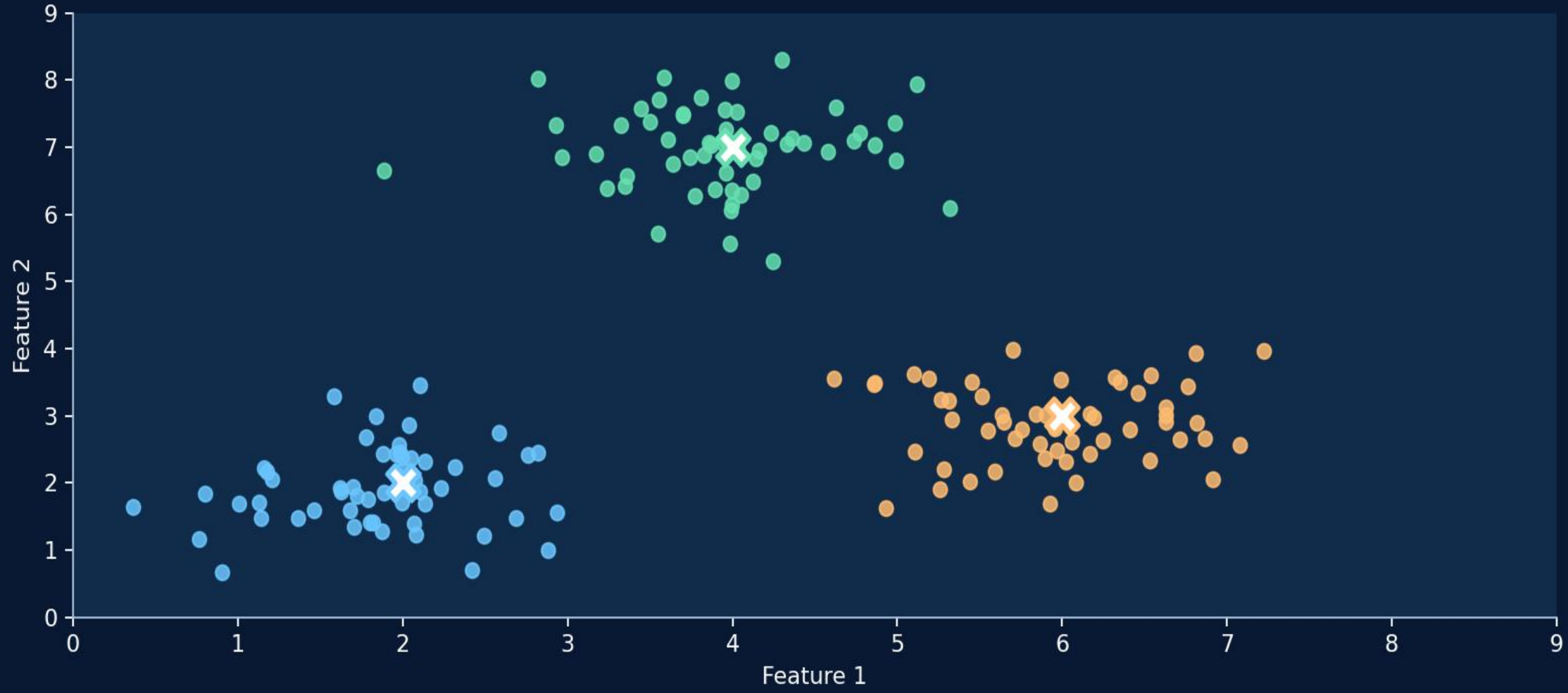
Supervised learning: labels guide prediction

Input features and known targets are used to learn a predictive boundary



Unsupervised learning: clusters emerge

There are no labels: the algorithm discovers groups from similarity



Reinforcement learning: action and feedback

An agent learns a policy by maximizing cumulative reward



Next observation + reward are returned to the agent

Example: maintenance now (−2) vs failure later (−10)

One industrial problem, three learning tasks

The same machine data can support very different AI objectives

SUPERVISED	Temperature, vibration + failure labels	**Predict: Will this machine fail?**
UNSUPERVISED	Temperature, vibration without labels	**Discover: Which operating modes exist?**
REINFORCEMENT	Machine state + action + reward	**Decide: When should we maintain it?**

Next: classical supervised methods → regression and classification

Regression vs classification

REGRESSION

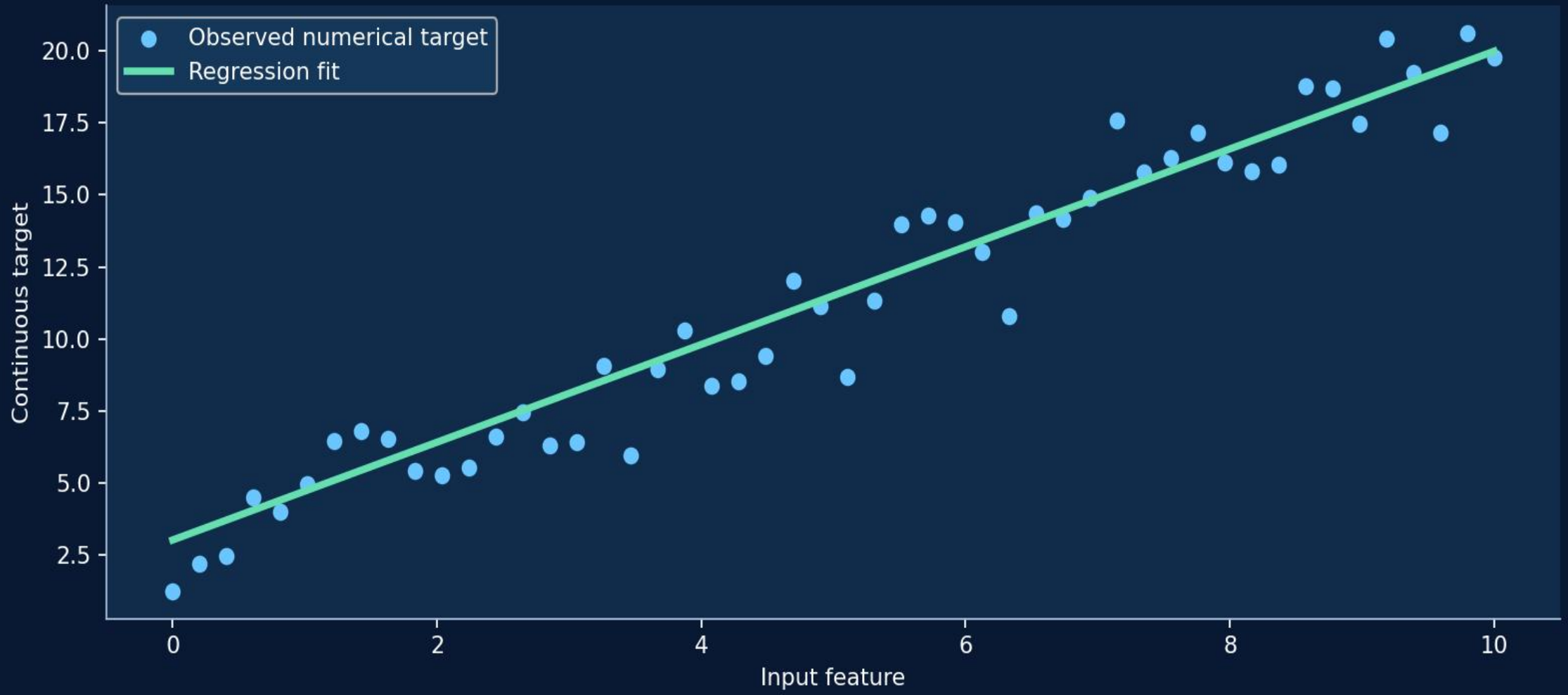
- Predicts a continuous number
- Energy demand, price, lifetime
- Metrics: MAE, RMSE, R^2

CLASSIFICATION

- Predicts a class or probability
- Failure, fraud, spam
- Metrics: precision, recall, F1

Regression vs classification: two different outputs

Regression predicts a number; classification predicts a class or probability



Linear regression

$$\hat{y} = wx + b$$

SYMBOLS & MEANING

- x: input feature
- w: slope or learned weight
- b: intercept or baseline
- ŷ: predicted numerical target

WORKED INTERPRETATION

Example: consumption = 5 × temperature + 100. At 30°C, predicted electricity consumption is 250 kWh.

Linear regression loss: MSE

$$\mathrm{MSE} = (1/n)\,\Sigma_i\,(y_i - \hat{y}_i)^2$$

SYMBOLS & MEANING

- n: number of examples
- y_i: actual measured target
- $\hat{y}_i$: model prediction
- Square penalizes larger errors more

WORKED INTERPRETATION

A prediction error of 10 contributes 100 squared units; an error of 2 contributes only 4. Training minimizes the average squared error.

Python: linear regression

```
import numpy as np
from sklearn.linear_model import LinearRegression

X = np.array([15, 20, 25, 30, 35])
X = X.reshape(-1, 1)
y = np.array([174, 201, 224, 251, 278])

model = LinearRegression()
model.fit(X, y)
print(model.coef_, model.intercept_)
print(model.predict([[32]]))
```

WHAT THE CODE DOES

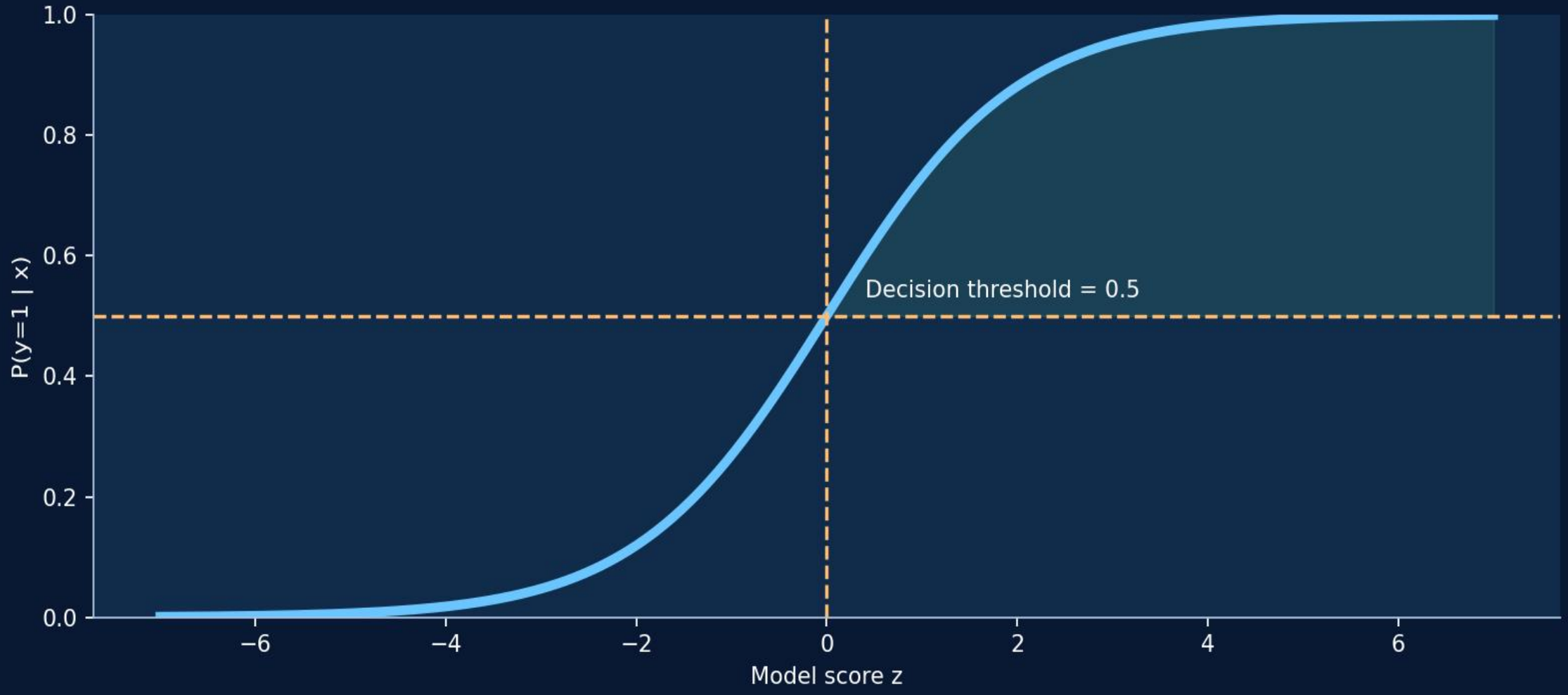
reshape(-1,1) makes a 2D feature matrix.

fit() estimates the slope and intercept.

predict() applies the learned line to a new temperature.

Sigmoid: from model score to probability

Logistic regression transforms any score into a probability between 0 and 1



Logistic regression

$$P(y = 1 \mid x) = \sigma(w^T x + b)$$

SYMBOLS & MEANING

- $y=1$: positive class (failure)
- $w^T x+b$: linear score
- σ: sigmoid mapping score to (0,1)
- Output: estimated class probability

WORKED INTERPRETATION

Unlike linear regression, logistic regression transforms the linear score to a bounded probability. A classification decision still needs a threshold.

The sigmoid function

$$\sigma(z) = 1 / (1 + \exp(-z))$$

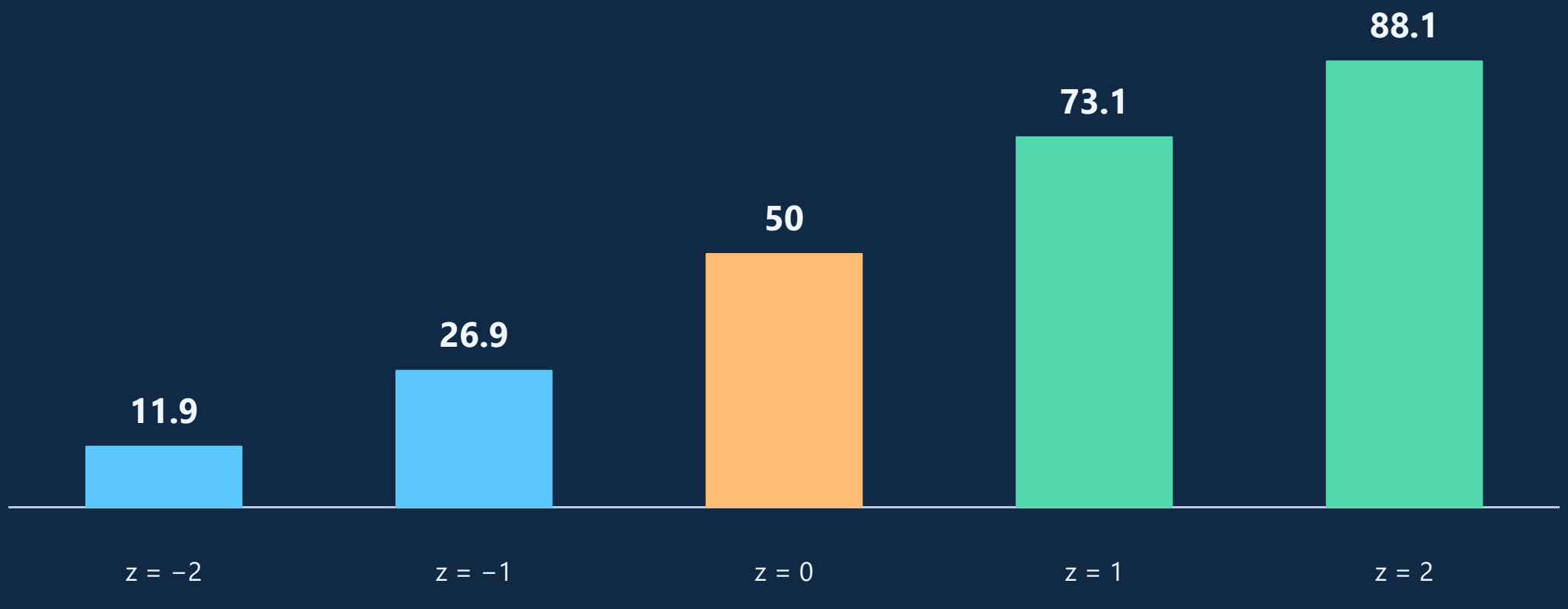
SYMBOLS & MEANING

- z: unbounded model score
- exp: exponential function
- $\sigma(0)=0.5$
- Large positive z → probability near 1
- Large negative z → probability near 0

WORKED INTERPRETATION

At $z=2$, sigmoid ≈ 0.881. At $z=-2$, sigmoid ≈ 0.119. The curve is smooth and S-shaped.

How sigmoid converts scores



Bars show sigmoid probabilities as percentages.

Binary classification threshold

$$\text{Predict failure if } P(\text{failure} \mid x) \geq \tau$$

SYMBOLS & MEANING

- τ: decision threshold
- $P(\text{failure}|x)$: model-estimated probability
- Default $\tau=0.5$ is not always optimal
- Lower τ generally increases sensitivity

WORKED INTERPRETATION

The threshold is a decision policy, not a fixed property of logistic regression. In high-cost failure scenarios, τ may be 0.2 or lower.

Python: logistic regression

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

model = make_pipeline(
    StandardScaler(),
    LogisticRegression()
)
model.fit(X_train, y_train)

p = model.predict_proba(X_test)[:, 1]
alerts = (p >= 0.20).astype(int)
```

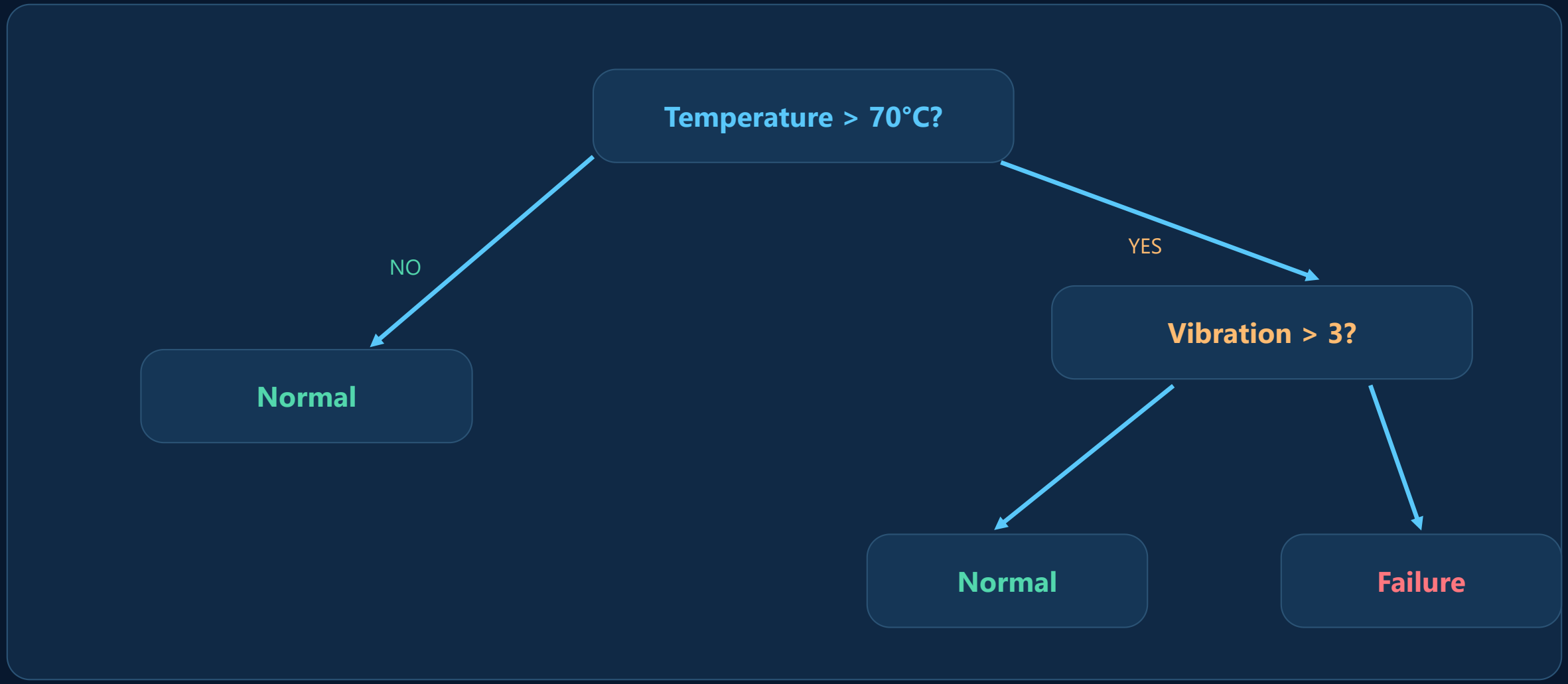
WHAT THE CODE DOES

StandardScaler learns scaling from training data inside the pipeline.

LogisticRegression estimates class probabilities.

The 0.20 threshold converts risk estimates into alerts.

Decision trees: transparent if–then logic



Decision tree: strengths and limitations

STRENGTHS

- Interpretable conditional rules
- Models nonlinear relationships
- Little preprocessing required

LIMITATIONS

- Deep trees can overfit
- Sensitive to small data changes
- A single tree can have high variance

Random forest: multiple trees

01

Bootstrap

Sample training rows

02

Random features

Consider feature subsets

03

Train trees

Fit many diverse trees

04

Aggregate

Vote or average outputs

Ensembling usually reduces variance compared with a single deep tree.

Decision tree vs random forest

DECISION TREE

- One transparent tree
- Fast to explain
- Often higher variance

RANDOM FOREST

- Many diverse trees
- Usually stronger generalization
- Less directly interpretable

Python: compare classifiers

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

models = [
 DecisionTreeClassifier(max_depth=4),
 RandomForestClassifier(n_estimators=100)
]
for model in models:
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    print(accuracy_score(y_test, pred))
```

WHAT THE CODE DOES

Both models use the same train/test split.

Compare unseen-data performance, not only training accuracy.

For rare failures, also inspect recall and precision.

Unsupervised learning

- There are no target labels in the training examples
- The algorithm searches for structure or similarity
- Common tasks: clustering, dimensionality reduction, anomaly detection
- Clusters are not automatically meaningful business categories

KEY INSIGHT

K-Means discovers groups; people still need to interpret what the groups mean.

K-Means: intuitive algorithm

01

Initialize

Choose K centroids

02

Assign

Find nearest centroid

03

Update

Recalculate cluster means

04

Repeat

Stop when stable

Goal: minimize within-cluster squared distances.

K-Means objective

$$J = \sum_k \sum_{x \in C_k} ||x - \mu_k||^2$$

SYMBOLS & MEANING

- K: number of clusters
- C_k: points assigned to cluster k
- μ_k: centroid of cluster k
- $||x-\mu_k||^2$: squared distance

WORKED INTERPRETATION

Points close to their assigned center contribute a small cost. Scaling features matters when their units differ greatly.

Python: K-Means segmentation

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

X = [[1,120],[2,150],[3,190],
     [5,480],[6,530],[7,810],
     [8,850],[9,920]]
X_scaled = StandardScaler().fit_transform(X)

km = KMeans(n_clusters=3,
            random_state=42, n_init=10)
labels = km.fit_predict(X_scaled)
print(labels)
```

WHAT THE CODE DOES

Each row is [purchase frequency, spending].

Scaling prevents spending from dominating distance.

Cluster numbers have no inherent order or meaning.

From ML prediction to rational action

01

Measure

Temperature and vibration

02

Predict

P(failure | sensors)

03

Cost model

Failure vs maintenance costs

04

Threshold

Compare probability to cost ratio

05

Act

Maintain or continue

Prediction ≠ decision. The action depends on predicted risk AND consequences.

Integrated rational ML decision

$$a^* = \arg\min_a \Sigma_o P(o \mid x, a)\, C(o, a)$$

SYMBOLS & MEANING

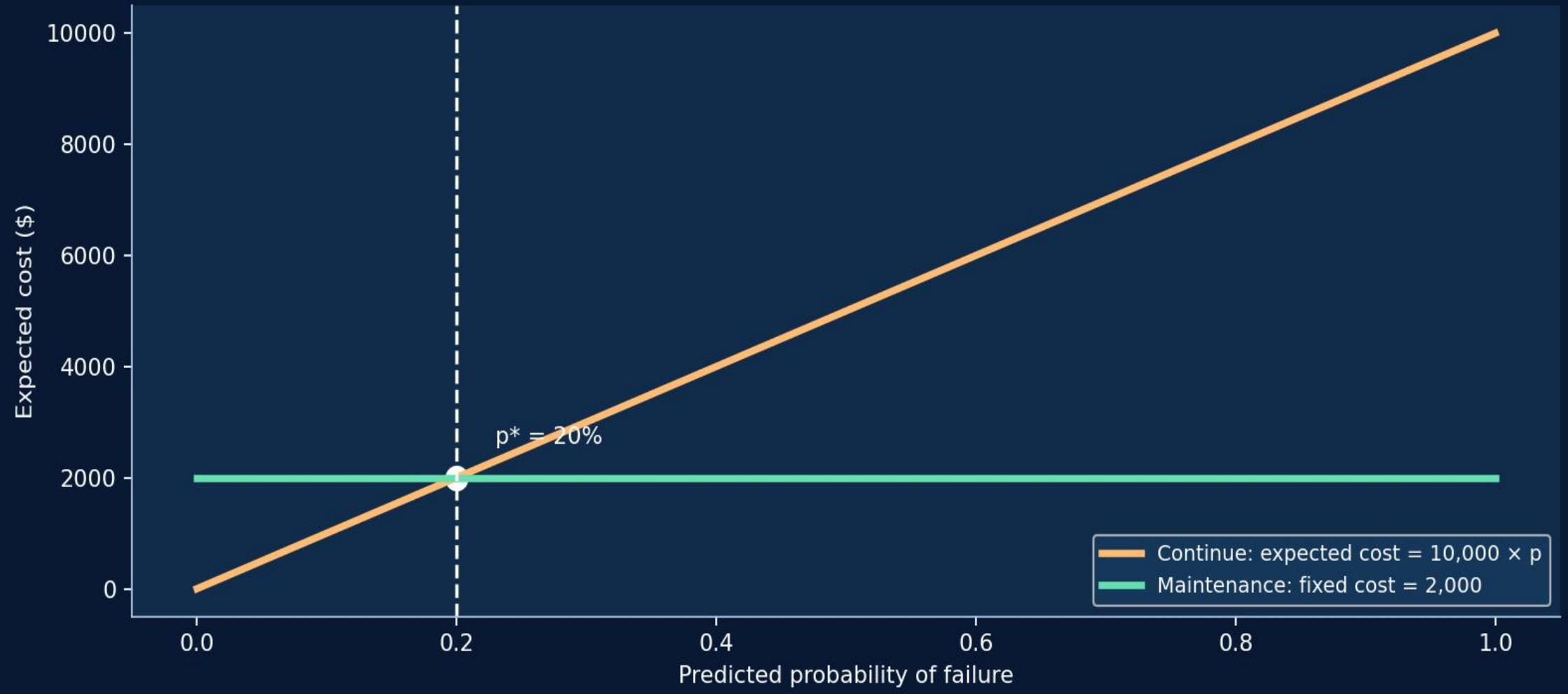
- x: observed machine features
- a: possible maintenance action
- o: future outcome
- P(o|x,a): outcome probability under action
- C(o,a): consequence/cost

WORKED INTERPRETATION

For each action, calculate expected cost across possible outcomes. Choose the feasible action with minimum expected cost.

A probability becomes a decision threshold

Maintenance is optimal when expected failure cost exceeds preventive maintenance cost



Python: ML-driven maintenance

```
p_failure = model.predict_proba(
    [[78, 4.2]]
)[0, 1]

maintenance_cost = 2000
failure_cost = 10000

cost_continue = p_failure * failure_cost
cost_maintain = maintenance_cost

choice = ('Maintain' if cost_maintain
          < cost_continue else 'Continue')
print(p_failure, choice)
```

WHAT THE CODE DOES

The classifier provides a probability.

The decision model multiplies that probability by the failure cost.

The maintenance action is selected when its cost is lower.

High accuracy ≠ good decisions

MODEL A

- Accuracy 95%
- Failure recall 60%
- More missed failures

MODEL B

- Accuracy 92%
- Failure recall 90%
- Fewer missed failures

Accuracy and recall

$$\text{Accuracy} = (TP + TN) / N \quad | \quad \text{Recall} = TP / (TP + FN)$$

SYMBOLS & MEANING

- TP: correctly detected failures
- TN: correctly detected normal cases
- FN: missed actual failures
- N: total observations

WORKED INTERPRETATION

If only 2% of observations are failures, a model predicting “normal” always achieves 98% accuracy but 0% failure recall.

Precision and F1

$$\text{Precision} = TP / (TP + FP) \quad | \quad F1 = 2PR/(P+R)$$

SYMBOLS & MEANING

- FP: false alarms
- Precision: how many alarms are correct
- Recall: how many failures are detected
- F1: harmonic mean of precision and recall

WORKED INTERPRETATION

Precision matters when false alarms are expensive; recall matters when missed failures are dangerous. F1 balances both, but not explicit costs.

Reliable model evaluation

01

Train split

Learn parameters

02

Validation

Tune model and threshold

03

Test split

Estimate final performance

04

Calibrate

Check predicted probabilities

A well-calibrated 70% prediction should correspond to ~70% observed frequency in comparable cases.

Risks and limitations

- Overfitting: learns noise rather than general patterns
- Data leakage: uses unavailable future information
- Distribution shift: new conditions differ from training
- Calibration error: predicted risks misrepresent real frequencies

KEY INSIGHT

For rational decisions, trustworthy probabilities can matter more than raw classification accuracy.

EXERCISE

Practice question: robot route

$$EU(A) = 0.8(100) + 0.2(-50) = 70$$

SYMBOLS & MEANING

- Route A success = 80%
- Success utility = +100
- Failure utility = −50
- Route B: 95% success, +80/−20

WORKED INTERPRETATION

Calculate $EU(B) = 0.95 \times 80 + 0.05 \times (-20) = 75$. Route B has higher expected utility despite a smaller success reward.

EXERCISE

Practice question: maintenance

$$0.15 \times \$20{,}000 = \$3{,}000 > \$2{,}000$$

SYMBOLS & MEANING

- Failure probability = 15%
- Failure loss = $20,000
- Maintenance cost = $2,000
- Maintenance assumed fully preventive

WORKED INTERPRETATION

Expected cost of continuing is $3,000, so preventive maintenance saves $1,000 in expected cost.

SUMMARY

Key synthesis

01

Data

What do we observe?

→

02

Probability

What may happen?

→

03

Utility

How costly is each outcome?

→

04

Decision

Which action is best?

Good AI systems combine predictive learning with rational action selection.

Final discussion

- Can a rational decision have a bad outcome? Why?
- Why can a 98%-accurate model be useless for detecting rare failures?
- How does a maintenance cost change the optimal threshold?
- When should we use regression, classification, or clustering?

KEY INSIGHT

Independent study: build a failure classifier and compare 50% vs cost-optimal decision thresholds.